

Data-Size vs. Model Complexity

Classical ML, Neural Networks, and Transfer Learning

A Comparative Study Across Tabular and Image Data

Author: Amir Talebi

Date: 9/18/2026

1. Project Objective

This project investigates a single central question: how does model performance scale with training-set size, and at what point (if any) does added model complexity actually pay off? To answer this in a controlled way, the same experiment (small / medium / large training-set size, with validation and test sets held fixed) was run across two very different data modalities: tabular data (UCI Adult Census Income) using Logistic Regression, polynomial-feature Logistic Regression, and a regularized Keras neural network; and image data (CIFAR-10) using a CNN trained from scratch and transfer learning from a frozen, ImageNet-pretrained backbone. Alongside accuracy, every experiment also tracked training cost, since a fair comparison of model complexity has to weigh accuracy gains against the compute they cost.

- **Datasets:** UCI Adult Census Income (48,842 rows, binary income target, ~24/76 class imbalance) and CIFAR-10 (60,000 images, 10 balanced classes).
- **Model ladder:** Logistic Regression → Logistic Regression + polynomial features → from-scratch NumPy neural net (sanity check) → regularized Keras neural network, and separately, CNN-from-scratch → frozen pretrained CNN → fine-tuned pretrained CNN.
- **Workflow discipline:** fixed train/validation/test splits, cross-validated hyperparameter tuning, early stopping, and a test set touched only once, at the very end, for the single best classical model.

2. Key Results: Tabular Data (Adult Census Income)

Every tabular model was scored on validation F1 (the primary metric, given the ~24/76 class imbalance) and validation ROC-AUC, at three training sizes: small ($n = 800$), medium ($n = 6,000$), and large ($n = 34,187$, the full pool).

Model	Size	n_train	Val F1	Val ROC-AUC	Train time (s)
Logistic Regression	small	800	0.623	0.892	2.57
Logistic Regression + Poly	small	800	0.632	0.888	0.13
Neural Net (Keras)	small	800	0.624	0.885	10.1
Logistic Regression	medium	6,000	0.649	0.903	2.15
Logistic Regression + Poly	medium	6,000	0.662	0.907	0.78
Neural Net (Keras)	medium	6,000	0.645	0.904	14.8
Logistic Regression	large	34,187	0.642	0.905	1.32
Logistic Regression + Poly	large	34,187	0.665	0.911	3.94
Neural Net (Keras)	large	34,187	0.641	0.907	44.9

Table 1. Full validation results, classical ML vs. neural network, tabular data.

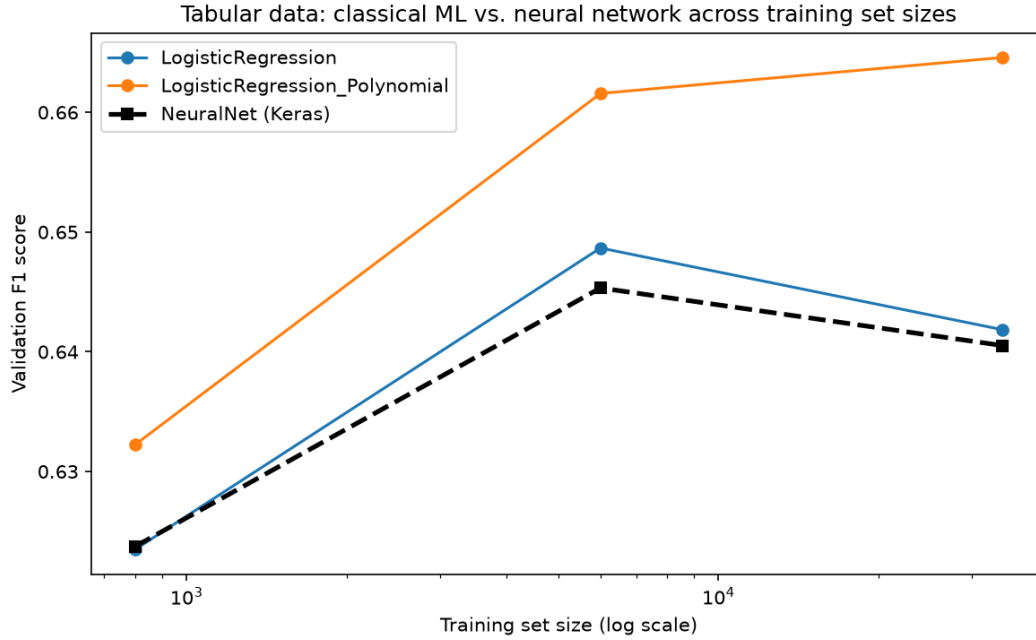


Figure 1. Validation F1 vs. training-set size (the project's master chart for the tabular track).

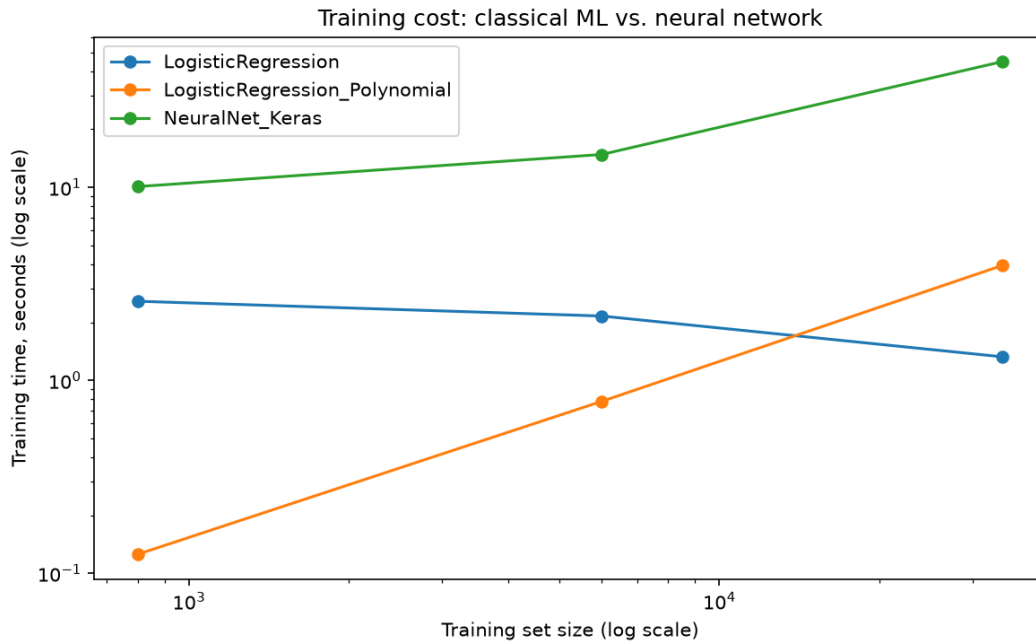


Figure 2. Training time vs. training-set size (log scale) (classical ML vs. neural network).

Across all three sizes, Logistic Regression with degree-2 polynomial features on the numeric columns is the best or tied-best tabular model, and it beats the Keras neural network at every size while training 5-30x faster. The plain logistic regression and the neural network track each other closely throughout, and the polynomial model's advantage over the neural network actually widens as more data becomes available, not the other way around.

3. Key Results: Image Data (CIFAR-10)

For the image track, a compact CNN trained from scratch was compared against transfer learning from a frozen, ImageNet-pretrained MobileNetV2 backbone, evaluated on validation accuracy (appropriate here since CIFAR-10's 10 classes are balanced).

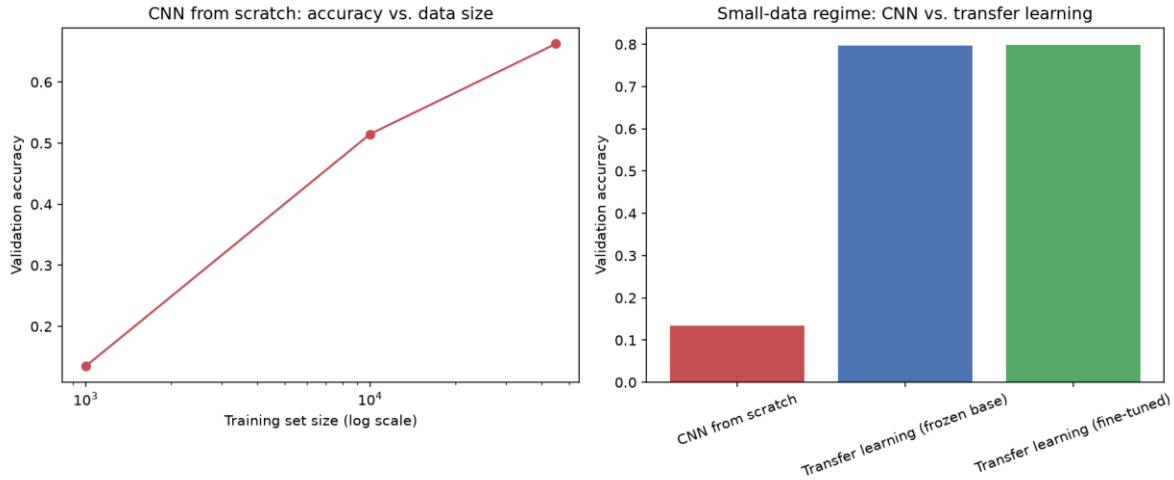


Figure 3. Left: CNN-from-scratch accuracy vs. training-set size. Right: CNN vs. transfer learning on the same $\sim 1,000$ -image small subset.

Approach	Training images	Val accuracy
CNN from scratch	1,000	0.135
CNN from scratch	10,000	0.515
CNN from scratch	45,000	0.663
Transfer learning (frozen base)	1,000	0.798
Transfer learning (fine-tuned)	1,000	0.799

Table 2. CNN accuracy across data sizes, and the transfer-learning comparison on the small subset.

The from-scratch CNN is far more data-hungry than the tabular models: validation accuracy roughly quadruples ($0.135 \rightarrow 0.663$) between 1,000 and 45,000 training images. On the identical $\sim 1,000$ -image subset, however, a frozen ImageNet-pretrained backbone reaches 79.8% accuracy. A roughly six-fold improvement over the from-scratch CNN's 13.5%, using only $\sim 12,800$ trainable parameters and no additional labeled data. Fine-tuning the last 20 layers nudges this only marginally further, to 79.9%.

4. Discussion: What the Results Mean

- **More complexity did not pay off on tabular data.** A well-regularized linear model with simple feature engineering matched or beat a deeper neural network at every training size, at a fraction of the training cost.
- **Regularization matters most when data is scarce.** A dropout/L2 sweep on the small tabular subset showed a nearly 15-point F1 gap between the best-regularized configuration ($F1 = 0.631$) and an unregularized one ($F1 = 0.479$), a clean illustration of the bias–variance trade-off.
- **CNNs are far more data-hungry than the tabular models.** The tabular models were nearly flat past a few thousand rows, while the from-scratch CNN kept climbing steeply all the way to the largest size tested.

- **Transfer learning was the single highest-leverage technique in the project.** It turned a barely-better-than-random image classifier into a strong one using the same small labeled set. A bigger jump than anything gained by adding model complexity elsewhere in the project.
- **The from-scratch NumPy neural network is validated.** Its training cost decreased smoothly, and its accuracy landed in the same range as scikit-learn's MLPClassifier on identical data. Good evidence the hand-derived forward/backward-propagation math was implemented correctly.

5. Limitations

- Single random seed throughout: all splits, subsamples, and initializations used the same seed, so reported numbers are point estimates without confidence intervals.
- Modest hyperparameter search budget for the tabular NN (5 combinations, one subset) and a compact, unaugmented CNN architecture (due to time limits).

6. Conclusions

For this tabular dataset, a well-regularized, feature-engineered linear model was the most practically attractive choice, beating a comparable neural network on both accuracy and training cost at every size tested. For image classification with limited labeled data, the highest-impact lever was not a bigger from-scratch model but transfer learning from a pretrained backbone, which converted a near-random classifier into a strong one with no extra labeled data. Together, the two tracks show two different data-scaling regimes side by side: the tabular models here are largely data-size-insensitive past a few thousand rows, while the from-scratch CNN is strongly data-size-sensitive.

7. Reflection Questions

Q1. At small training set sizes, did classical ML or the neural network perform better?

Classical ML performed slightly better. At the small size ($n = 800$, leftmost points on the master chart, Figure 1), Logistic Regression with polynomial features reached $F1 = 0.632$ versus $F1 = 0.624$ for the Keras neural network. A small but consistent edge for the classical model, achieved at roughly 1/80th of the training time (0.13 s vs. 10.1 s, Table 1).

Q2. Did the gap between classical ML and the neural network shrink, stay flat, or reverse as training set size grew? What does that suggest about each model class's capacity and data efficiency?

The gap widened in favor of classical ML: the $F1$ advantage of polynomial Logistic Regression over the neural network grew from +0.008 at small, to +0.017 at medium, to +0.024 at large. This is the opposite of the usual expectation that neural networks pull ahead with more data. It suggests that for this feature set, a degree-2 polynomial expansion already captures most of the exploitable non-linear structure, so the network's extra capacity is not needed. And because the network is also harder to regularize well at every size, it never converts that unused capacity into an accuracy advantage.

Q3. Was the compute cost trade-off (training time) worth any accuracy gained by the more complex model? At what data size, if any, did the neural network's extra cost become “worth it”?

No. At no training size did the neural network's extra cost pay for itself. Figure 2 shows the neural network took 10–45 seconds to train versus 0.1–4 seconds for the polynomial Logistic Regression, and Table 1 shows the network's F1 was lower than the polynomial model's at every size (small, medium, and large alike). Since accuracy never favored the more expensive model, there was no data size in this experiment where the added compute was justified.

Q4. For the image classification track: How much did transfer learning help in the small-data regime, and does that match the theoretical explanation (reuse of general visual features learned from ImageNet)?

Transfer learning provided a dramatic improvement: on the identical ~1,000-image training set, the from-scratch CNN reached only 13.5% validation accuracy, barely above the 10% random baseline for 10 classes, while the frozen ImageNet-pretrained backbone reached 79.8% (Figure 3, right panel; Table 2). Roughly a six-fold gain using only about 12,800 trainable parameters. This matches the theoretical explanation well: the pretrained backbone had already learned general, transferable visual features (edges, textures, shapes) from millions of ImageNet images, so the small CIFAR-10 subset only had to train a lightweight classification head on top of those features rather than learn visual representations from nothing. Fine-tuning the last 20 layers added almost nothing further (79.8% → 79.9%), suggesting the frozen features were already close to as useful as they were going to get with this amount of fine-tuning data.

Q5. What would I do next with more time?

With more time, the next steps would be to add more datasets across both modalities to see whether the “classical ML wins on tabular data” and “transfer learning wins on small image data” findings hold beyond these two specific datasets; run each experiment across multiple random seeds and report confidence intervals or statistical-significance tests rather than single point estimates; carry out a more thorough hyperparameter search (layer sizes, learning rates, batch sizes) for the tabular neural network; and explore deeper or augmented CNN/transfer-learning architectures, along with more fine-tuning data, to see whether they close the gap with classical ML on tabular data or push transfer-learning accuracy higher on images. I would also try different combinations of hyperparameters for the fine-tuning the last 20 layers part where I think there's still room for improvement.